

**LAPORAN IMPLEMENTASI STRUKTUR DATA TERPADU
(QUEUE, STACK, BST, DAN GRAPH) PADA SMART
LIBRARY MANAGEMENT & RECOMMENDATION SYSTEM
MATA KULIAH ALGORITMA DAN STRUKTUR DATA**



Dosen Pengampu: Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.

Disusun oleh Kelompok 7:

Yossan Bagus Pramuditya	25051030013
Bimo Aqces Saefullah	25051030003
Muhammad Erji Alvinandra Pratama	25051030008
Daffa Rifqi Mu'afa	25051030019

**PROGRAM STUDI TEKNIK ELEKTRO
DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA**

2026

I. LATAR BELAKANG

Perpustakaan dalam lingkungan institusi akademik, khususnya Universitas bukan sekadar tempat penyimpanan buku, melainkan pusat sirkulasi pengetahuan yang mendukung jalannya perguruan tinggi. Seiring dengan pesatnya pertumbuhan jumlah koleksi buku dan transaksi harian di perpustakaan, manajemen perpustakaan dengan metode konvensional mulai menghadapi tantangan digitalisasi.

Sistem informasi perpustakaan modern dituntut tidak hanya mampu mencatat data peminjaman secara statis, tetapi juga harus adaptif, responsif, dan cerdas dalam memberikan layanan kepada para pembaca. Salah satu fitur yang dicari pada platform modern adalah fitur untuk merekomendasikan buku berdasarkan perilaku pembaca, mirip dengan teknologi yang diterapkan pada industri e-commerce skala besar.

Namun, implementasi perpustakaan modern ini sering mengalami masalah efisiensi komputasi pada lapisan arsitektur data internal. Penggunaan struktur data linear standar bawaan bahasa pemrograman sering kali memicu penurunan performa ketika volume data melonjak. Sebagai contoh, pencarian buku berdasarkan nomor ISBN pada daftar linier akan memakan waktu operasi $O(n)$, yang dimana sistem akan semakin lambat seiring bertambahnya koleksi. Masalah lainnya adalah penumpukan riwayat transaksi global yang tidak terkelola, antrean pemesanan buku populer yang tidak adil bagi anggota premium, serta ledakan komputasi kuadratik $O(n^2)$ saat graf rekomendasi mencoba mencari keterkaitan antar buku.

Untuk mengatasi permasalahan tersebut, proyek ini mengimplementasikan Smart Library Management & Recommendation System tanpa bergantung pada pustaka (library) struktur data pihak ketiga. Arsitektur katalog buku dioptimalkan menggunakan Binary Search Tree (BST) berbasis string ISBN untuk memangkas waktu pencarian dari linear menjadi logaritmik $O(\log n)$. Alur antrean pemesanan dikelola secara adil dengan opsi antrean prioritas (Priority Queue), sementara tumpukan log aktivitas sirkulasi diamankan melalui implementasi Stack berbasis Linked List yang dilengkapi dengan pengarsipan agar penggunaan memori tetap efisien dan konstan.

Poin keunggulan utama dari sistem ini terletak pada pemanfaatan struktur data Graf tak berarah (Undirected Graph) dengan representasi daftar ketetanggaan (Adjacency List) untuk memetakan hubungan buku yang sering dipinjam bersamaan. Proses ekstraksi rekomendasi menggunakan algoritma Breadth-First Search (BFS) dengan pembatasan kedalaman maksimal dua tingkatan ($\text{hop} \leq 2$), yang kemudian disaring menggunakan filter bobot frekuensi minimum. Melalui kombinasi struktur data mandiri ini, diharapkan Smart Library Management & Recommendation System ini mampu menyajikan performa pencarian dan rekomendasi yang cepat, efisien, serta memiliki skalabilitas untuk menghadapi pertumbuhan data.

II. ARSITEKTUR SISTEM

Sistem Smart Library Management & Recommendation System ini dirancang menggunakan pendekatan Arsitektur Berlapis, yang dimana terbagi ke dalam tiga tingkatan utama. Pendekatan ini dipilih agar logika penyimpanan data murni dipisahkan dari logika bisnis aplikasi, sehingga mempermudah proses perawatan kode (maintenance) dan pengujian unit (unit testing).

Lapisan terluar dari sistem ini Lapisan Antarmuka, yang dimana lapisan ini adalah sistem yang berinteraksi langsung dengan pengguna, baik itu untuk petugas perpustakaan maupun anggota. Lapisan ini diimplementasikan di dalam berkas `main.py` yang bertugas menyediakan menu berbasis teks CLI (Command Line Interface) yang interaktif, sistem ini menangani masukan (input) dari keyboard, serta menampilkan output laporan transaksi yang dilakukan dan daftar rekomendasi buku ke layar terminal.

Lapisan menengah dari sistem ini adalah Lapisan Logika Perpustakaan, yang dimana lapisan ini memproses segala aturan fungsional perpustakaan. Lapisan ini direpresentasikan oleh modul-modul di dalam folder `src/modules/`, seperti `layanan_katalog.py` untuk alur registrasi buku, `layanan_transaksi.py` untuk memproses peminjaman, pemesanan, dan pengembalian, serta objek Models yang membungkus entitas data. Lapisan ini bertindak sebagai jembatan yang mengambil

data dari antarmuka, mengolahnya, lalu menyimpannya ke dalam struktur data yang sesuai.

Lapisan paling dalam dari sistem ini adalah Lapisan Struktur Data Inti, yang dimana lapisan ini menjadi pondasi penyimpanan data di dalam RAM laptop selama program berjalan. Oleh karena itu, lapisan ini berisi seluruh kelas struktur data murni tanpa pustaka (library) struktur data pihak ketiga di folder `src/data_structures/`, meliputi BSTKatalog untuk pencarian buku, Stack untuk log riwayat transaksi global, Queue untuk antrean pemesanan buku populer, dan GraphRekBuku untuk memetakan hubungan keterkaitan antar buku, yang digunakan untuk fitur rekomendasi.

Ketika sebuah aksi dipicu oleh pengguna di halaman utama `main.py`, yaitu Lapisan Antarmuka yang merupakan lapisan terluar dari sistem ini akan menyediakan menu berbasis teks CLI (Command Line Interface) yang interaktif, aliran data akan mengalir secara sekuensial melewati lapisan-lapisan tersebut. Sebagai contoh, ketika fungsi rekomendasi buku dijalankan, lapisan ini akan mengirimkan string ISBN buku yang sedang dilihat ke lapisan menengah, yaitu Lapisan Logika Perpustakaan.

Kemudian, fungsi di dalam `layanan_katalog.py` akan memanggil objek graf GraphRekBuku yang berada di lapisan terdalam, yaitu Lapisan Struktur Data Inti. Algoritma Breadth-First Search (BFS) internal graf akan menelusuri daftar ketetanggaan (Adjacency List) hingga 2 hop, kemudian menyaring hasilnya berdasarkan bobot frekuensi, lalu mengembalikan daftar objek buku yang relevan kembali ke lapisan terluar hingga akhirnya dicetak di terminal pengguna.

III. PENJELASAN ALGORITMA

Struktur data yang dibuat pada Smart Library Management & Recommendation System ini tidak menggunakan pustaka (library) struktur data pihak ketiga, dengan kata lain Linked List, Stack, Queue, Tree, dan Graph diimplementasikan dari nol.

1. Binary Search Tree (BST)

Pada file `bst.py`, yaitu struktur data Binary Search Tree (BST), setiap ada setiap buku baru, data dari buku tersebut akan dibungkus dalam satu simpul (node) pada Binary Search Tree (BST), data tersebut terdiri atas judul, pengarang, kategori, status, dan ISBN sebagai key pada Binary Search Tree (BST). Cabang dari setiap node ini berupa `self.left` untuk menampung key berupa ISBN yang lebih kecil dan `self.right` untuk menampung key berupa ISBN yang lebih besar. Operasi mendaftarkan sebagai node Binary Search Tree (BST) ini memiliki kompleksitas $O(1)$.

Pada operasi penyisipan (insert), operasi tersebut dijalankan secara rekursif, yang dimana sistem akan membandingkan string ISBN yang baru disisipkan saat operasi membatalkan tindakan penghapusan buku yang rusak dengan node yang ada saat ini secara alfabet, karena ISBN sebagai key berupa string. ISBN yang baru disisipkan tersebut akan bergerak ke kiri atau kanan hingga menemukan ruang kosong untuk meletakkan node yang baru disisipkan tersebut. Selain itu, operasi insert ini juga berfungsi untuk membuat 80 katalog buku dalam memory Binary Search Tree (BST). Operasi insert pada Binary Search Tree (BST) ini memiliki kompleksitas $O(\log n)$ untuk average, dan $O(n)$ untuk worst case.

Pada operasi pencarian (search), operasi ini menggunakan prinsip yang mirip seperti operasi penyisipan, yang dimana jika ISBN sebagai key yang dicari lebih kecil dari node target, maka operasi pencarian akan bergeser ke sub-tree kiri, sedangkan jika ISBN sebagai key yang dicari lebih besar dari node target, maka operasi pencarian akan bergeser ke kanan. Oleh karena itu, operasi ini akan memangkas ruang pencarian menjadi hampir setengahnya di setiap tingkatan. Operasi search pada Binary Search Tree (BST) ini memiliki kompleksitas $O(\log n)$ untuk average, dan $O(n)$ untuk worst case.

Pada operasi update, operasi ini akan memperbarui status dari buku di Binary Search Tree (BST), status tersebut dapat berupa tersedia yaitu "0" atau dipinjam yaitu "1". Fitur update ini diimplementasikan saat pengembalian buku, batalkan pemesanan, peminjaman buku, dan pemesanan buku. Operasi update pada

Binary Search Tree (BST) ini memiliki kompleksitas $O(\log n)$ untuk average, dan $O(n)$ untuk worst case.

Pada operasi delete, operasi ini akan menghapus buku dari Binary Search Tree (BST), fitur yang menggunakan operasi ini adalah fitur penghapusan buku yang rusak di perpustakaan. Operasi update pada Binary Search Tree (BST) ini memiliki kompleksitas $O(\log n)$ untuk average, dan $O(n)$ untuk worst case.

2. Stack

Pada file Stack.py, yaitu struktur data Stack berbasis Linked List, operasi yang digunakan adalah operasi push dan pop, yang dimana saat operasi push, data baru akan ditumpuk dan ditempatkan di bagian paling atas (top). Operasi push ini digunakan untuk peminjaman buku dan penghapusan buku yang rusak. Operasi push pada Stack ini memiliki kompleksitas $O(1)$.

Selain itu, terdapat operasi pop, yang dimana operasi ini digunakan untuk membatalkan tindakan terakhir, contohnya yaitu membatalkan penghapusan buku rusak dan peminjaman buku, hal ini dilakukan dengan mengambil data paling atas yang sudah ditempatkan oleh operasi pop sebelumnya. Operasi ini dapat dilakukan karena struktur data Stack menggunakan cara kerja Last In, First Out, yang dimana data yang dimasukkan saat terakhir akan dikeluarkan pertama kali. Operasi pop pada Stack ini memiliki kompleksitas $O(1)$.

3. Linked_List

Pada file queue_ll.py, yaitu struktur data Linked List, operasi yang digunakan adalah enqueue, dequeue, dan delete. Untuk operasi enqueue diterapkan pada fitur pemesanan buku yang sedang dipinjam pembaca lain, yang dimana pengunjung yang terakhir memesan buku akan ditempatkan di self.tail, sedangkan pengunjung di antrian paling depan yaitu self.head akan mendapatkan buku yang dipesan terlebih dahulu. Operasi enqueue pada Linked List ini memiliki kompleksitas $O(1)$.

Selain itu terdapat operasi dequeue, yang operasi ini diterapkan pada fitur mengembalikan buku, yang dimana saat buku dikembalikan oleh pengunjung di antrian paling depan, maka posisi self.head akan digantikan oleh pengunjung tepat di belakangnya. Operasi ini dapat dilakukan karena struktur data Linked List

menggunakan cara kerja First In, First Out, yang dimana data yang dimasukkan pertama kali maka akan dilayani pertama kali juga. Operasi enqueue pada Linked List ini memiliki kompleksitas $O(1)$.

Selain itu, sebagai tambahan disini juga terdapat operasi delete, yang dimana terdapat pada fitur fitur pembatalan pesanan, operasi ini akan menghapus pengunjung yang membatalkan pesannya pada antrian Linked List. Operasi enqueue pada Linked List ini memiliki kompleksitas $O(n)$.

4. Graph

Pada file graph.py, terdapat algoritma Breadth First Search (BFS), algoritma tersebut akan digunakan untuk menerapkan fitur rekomendasi dengan melihat hubungan buku maksimal 2 hop. Pada fitur rekomendasi, sistem akan memantau buku yang sedang dibaca pengguna, kemudian akan mengecek buku-buku yang pernah dipinjam bersamaan dengan buku tersebut (hop 1), lalu buku-buku tersebut akan melihat buku-buku lain yang pernah dipinjam bersamaan dengan buku-buku tersebut (hop 2). Buku buku yang dicek tersebut akan ditampilkan di terminal sebagai rekomendasi buku untuk dibaca selanjutnya untuk pembaca. Fitur rekomendasi graph dengan algoritma Breadth First Search (BFS) ini memiliki kompleksitas $O(V+E)$.

5. Sorting

Sorting pada sistem ini menggunakan Merge Sort dan Shell Sort, yang dimana Shell Sort akan digunakan untuk menemukan peminjaman dengan durasi terlama, sedangkan untuk Merge Sort digunakan untuk menemukan buku yang paling sering dipinjam. Merge Sort memiliki kompleksitas $O(n \log n)$ karena membelah data menjadi dua bagian secara konsisten hingga berukuran terkecil, lalu menggabungkannya kembali sambil mengurutkan elemen sepanjang data, sedangkan untuk Shell Sort yaitu memiliki kompleksitas $O(n^2)$ karena algoritma tersebut mengandalkan metode perbandingan berbasis jarak variabel (gap).

IV. ANALISIS BIG-O

4.1 Analisis Big-O pada Kode

Big-O adalah suatu metode yang digunakan untuk mengukur efisiensi dari suatu algoritma, yang dimana metode ini akan berfokus pada skenario terburuk, sesuai dengan dua golden rule big-O, yaitu abaikan konstanta dan ambil suku tertinggi. Pada analisis ini, akan dibahas mengenai analisis big-O pada fitur-fitur CLI di main.py:

1. BANTUAN

Memiliki kompleksitas $O(1)$, karena hanya berisi logika untuk input yang dihubungkan ke percabangan menggunakan if else.

2. CARI_BUKU

Memiliki kompleksitas $O(\log n)$ untuk average, dan $O(n)$ untuk worst case. Hal ini dikarenakan fitur pencarian menggunakan Binary Search Tree (BST), yang dimana akan $O(\log n)$ jika tree balance karena memotong setengah operasi pada setiap tingkatan, dan akan $O(n)$ saat tree skewed karena tidak memotong operasi apapun.

3. PINJAM

Memiliki kompleksitas $O(\log n+k)$ untuk average, dan $O(n+k)$ untuk worst case. Fitur ini memiliki prinsip yang sama seperti fitur nomor 2, yaitu tergantung Binary Search Tree (BST) balance atau skewed. Variabel n adalah operasi untuk search dan update di Binary Search Tree (BST), dan variabel k adalah jumlah riwayat peminjaman buku dari anggota yang nantinya akan dipakai untuk fitur rekomendasi.

4. KEMBALIKAN

Memiliki kompleksitas $O(\log n)$ untuk average, dan $O(n)$ untuk worst case. Hal ini dikarenakan fitur ini berbasis pada operasi search Binary Search Tree (BST), yang dimana kompleksitasnya akan akan bergantung pada apakah Binary Search Tree (BST) balance atau skewed.

5. PESAN

Memiliki kompleksitas $O(\log n)$ untuk average, dan $O(n)$ untuk worst case. Hal ini dikarenakan fitur ini masih berbasis pada operasi search Binary

Search Tree (BST), yang dimana kompleksitasnya akan akan bergantung pada apakah Binary Search Tree (BST) balance atau skewed.

6. ANTRIAN

Memiliki kompleksitas $O(n)$, karena untuk memantau antrian, Linked List harus menelusuri seluruh node dari head sampai tail untuk mengetahui daftar antrian.

7. REKOMENDASI

Memiliki kompleksitas $O(V+E)$, yang dimana V adalah variabel untuk jumlah vertex (titik), dan E adalah variabel untuk jumlah edge (garis yang menghubungkan antar titik). Algoritma ini memastikan setiap vertex dimasukkan ke dalam queue, dan setiap edge diperiksa untuk mengeksplorasi tetangga dari vertex tersebut

8. KATALOG

Memiliki kompleksitas $O(n)$, karena fitur ini berbasis operasi traversal pada Binary Search Tree (BST) yaitu inorder, yang dimana operasi ini perlu mengunjungi setiap node yang ada di Binary Search Tree (BST).

9. BATALKAN TERAKHIR

Memiliki kompleksitas $O(\log n)$ untuk average, dan $O(n)$ untuk worst case. Meskipun berbasis operasi undo pada Stack yang memiliki kompleksitas $O(1)$, fitur batalkan terakhir juga perlu mengupdate status buku ke Binary Search Tree (BST) saat pesanan buku dibatalkan, yang dimana operasi update tersebut memerlukan operasi search Binary Search Tree (BST), sehingga kompleksitas menjadi bergantung pada kondisi apakah Binary Search Tree (BST) balance atau skewed.

10. BATALKAN PESANAN

Memiliki kompleksitas $O(\log n)$ untuk average, dan $O(n)$ untuk worst case. Meskipun berbasis operasi undo pada Stack yang memiliki kompleksitas $O(1)$, fitur batalkan pesanan juga perlu mengupdate status buku ke Binary Search Tree (BST) saat pesanan buku dibatalkan, yang dimana operasi update tersebut memerlukan operasi search Binary Search Tree (BST), sehingga kompleksitas

menjadi bergantung pada kondisi apakah Binary Search Tree (BST) balance atau skewed.

11. HAPUS RUSAK

Memiliki kompleksitas $O(\log n)$ untuk average, dan $O(n)$ untuk worst case. Hal ini dikarenakan untuk menghapus buku yang rusak di Binary Search Tree (BST), operasi search harus dilakukan terlebih dahulu, sehingga kompleksitasnya bergantung pada kondisi apakah Binary Search Tree (BST) balance atau skewed.

12. LAPORAN BULANAN

Memiliki kompleksitas $O(n \log n + n^2)$ karena fitur ini menjalankan operasi Merge Sort yang memiliki kompleksitas $O(n \log n)$, dan Shell Sort menjalankan operasi $O(n^2)$. Kedua sorting tersebut digunakan untuk melakukan sorting pada fitur laporan bulan, yang dimana Merge Sort untuk sorting berdasarkan seberapa sering buku dipinjam, sedangkan Shell Sort untuk sorting berdasarkan durasi peminjaman.

13. KELUAR

Memiliki kompleksitas $O(1)$, karena hanya menjalankan logika untuk keluar dari program yang sedang dijalankan.

4.1 Pertanyaan Analisis

1. BST Katalog dengan kunci ISBN string: jika 80 buku disisipkan dalam urutan teratur leksikografis (ISBN-0001, ISBN-0002, ..., ISBN-0080), BST menjadi linear. Hitung tinggi tree worst-case vs rata-rata (data acak). Untuk $N=10.000$ judul perpustakaan universitas, berapa operasi search? Bandingkan dengan implementasi Hash Table $O(1)$ average dan jelaskan kapan BST tetap lebih unggul.

Jawab:

Jika 80 buku dimasukkan secara teratur leksikografis, maka BST tidak akan seimbang dan berubah menjadi struktur linear menyerupai linked list. Dalam kondisi ini, tinggi tree menjadi: $h = N-1$, yang dimana untuk 80 buku: $h = 80-1 = 79$. Artinya operasi pencarian pada worst-case dapat mencapai $O(n)$ (sekitar 80 langkah pencarian). Sebaliknya, jika data ISBN dimasukkan secara acak, BST

cenderung lebih seimbang. Tinggi rata-rata BST mendekati: $\log_2 n$, yang dimana untuk 80 buku: $\log_2 80 \approx 6.32$, sehingga rata-rata pencarian hanya membutuhkan sekitar 6–7 operasi atau langkah saja.

Untuk skala perpustakaan universitas dengan 10.000 judul, worst-case BST yaitu $O(n)$, yang dimana sekitar 10.000 operasi pencarian, sedangkan untuk average case BST: $\log_2 10000 \approx 13$, sehingga rata-rata pencarian hanya membutuhkan sekitar 13–14 operasi atau langkah saja.

Dibandingkan dengan Hash Table, pencarian ISBN pada Hash Table memiliki average case $O(1)$, sehingga secara lebih cepat untuk pencarian key, namun BST tetap memiliki keunggulan pada beberapa kondisi, diantaranya yaitu data otomatis tersusun teratur, mendukung traversal inorder, mendukung range query, dan mudah mencari ISBN minimum/maksimum.

2. Queue antrian pemesanan per-buku: jika sebuah buku populer memiliki 50 anggota dalam antrian dan buku dikembalikan rata-rata 3 hari sekali, estimasikan waktu tunggu anggota ke-50 dalam hari. Bandingkan dengan pendekatan Priority Queue (anggota premium didahulukan): hitung biaya tambahan operasi enqueue $O(n)$ vs queue biasa $O(1)$ untuk $N=50$ anggota.

Jawab:

Pada sistem ini, setiap buku memiliki queue FIFO menggunakan struktur linked list. Operasi enqueue dan dequeue berjalan dengan kompleksitas $O(1)$ karena enqueue langsung menambah di next tail, sedangkan dequeue langsung menghapus di head. Jika sebuah buku populer memiliki 50 anggota dalam antrian dan buku dikembalikan setiap 3 hari sekali, maka estimasi waktu tunggu anggota ke 50 adalah: $50 \times 3 = 150$ hari, karena setiap anggota harus menunggu anggota sebelumnya selesai meminjam.

Namun jika sistem diubah menjadi Priority Queue, misalnya anggota premium diprioritaskan, maka proses enqueue kompleksitasnya menjadi $O(n)$ karena sistem harus mencari posisi prioritas yang sesuai. Untuk $N = 50$ anggota, Queue biasa hanya membutuhkan 1 langkah insert, sedangkan Priority Queue dapat membutuhkan hingga 50 langkah pencarian posisi.

3. Graf Rekomendasi Ko-pinjam menggunakan BFS hop ≤ 2 . Jika setiap buku rata-rata terhubung ke 4 buku lain (degree=4) dan koleksi berisi 80 buku, estimasikan berapa node yang dijelajahi BFS dari satu sumber dalam 2 hop. Apakah semua rekomendasi relevan? Usulkan cara memfilter menggunakan bobot edge (frekuensi co-pinjam minimum) dan analisis dampaknya terhadap Big-O.

Jawab:

Kompleksitas BFS secara umum adalah $O(V+E)$, dengan V adalah jumlah vertex (buku), dan E adalah jumlah edge (hubungan co-pinjam). Jika setiap buku rata-rata terhubung ke 4 buku lain (degree = 4), maka estimasi node yang dijelajahi dari satu sumber adalah: Hop 1 = 1×4 node, Hop 2 = $4 \times 4 = 16$ node, Total: $1+4+16 = 21$ node. Jadi dari total 80 buku, BFS hanya menjelajahi sekitar 21 buku dalam 2 hop.

Namun tidak semua rekomendasi pasti relevan. Beberapa hubungan mungkin hanya terjadi satu kali sehingga hubungan antar buku lemah, sehingga untuk meningkatkan kualitas rekomendasi, sistem dapat menggunakan filtering berdasarkan bobot edge, misalnya hanya menampilkan relasi dengan frekuensi co-pinjam ≥ 3 , sehingga rekomendasi menjadi lebih relevan karena hanya hubungan kuat. Filtering tersebut tidak mengubah kompleksitas BFS, sehingga tetap $O(V+E)$, tetapi jumlah edge yang diproses menjadi lebih sedikit sehingga runtime menjadi lebih cepat.

4. Stack Riwayat Transaksi global menyimpan semua peminjaman/pengembalian. Jika perpustakaan memproses 200 transaksi/hari, berapa node Stack digunakan dalam satu bulan tanpa pembersihan? Usulkan strategi archiving: setelah N transaksi, pindahkan ke struktur lain (misalnya Linked List historis) agar Stack tetap ringkas. Analisis Big-O operasi archiving.

Jawab:

Jika perpustakaan memproses 200 transaksi per hari, maka dalam 30 hari jumlah node stack menjadi: $200 \times 30 = 6000$, sehingga tanpa pembersihan, stack dapat berisi sekitar 6000 node dalam satu bulan. Ukuran yang semakin membesar

ini mengakibatkan penggunaan memori meningkat, traversal riwayat menjadi lebih lambat, dan proses laporan bulanan menjadi lebih berat.

Untuk mengatasi hal tersebut, sistem dapat menerapkan strategi archiving, yang dimana jika jumlah transaksi aktif melebihi 1000, transaksi lama dipindahkan ke struktur lain seperti linked list arsip atau database/file. Dengan demikian stack utama dapat tetap melakukan operasi undo dengan cepat. Proses archiving memiliki kompleksitas $O(K)$, dengan K adalah jumlah transaksi yang dipindahkan, strategi ini membantu menjaga performa stack tetap stabil meskipun jumlah transaksi terus bertambah.

5. Sistem ini menggunakan BST untuk katalog dan Graf untuk rekomendasi.

Jelaskan bagaimana menambahkan fitur **PENCARIAN MULTI-KRITERIA** (misalnya: buku kategori 'Teknik' dengan status **TERSEDIA**, diurutkan berdasarkan judul). Apakah BST cocok untuk query ini?

Usulkan penambahan struktur data (misalnya secondary index) dan analisis kompleksitasnya.

Jawab:

BST pada sistem ini menggunakan ISBN sebagai key utama. Struktur ini sangat optimal untuk pencarian ISBN, karena memiliki kompleksitas $O(\log n)$, tetapi kurang cocok untuk query multi-kriteria karena BST hanya tersusun berdasarkan ISBN, bukan berdasarkan kategori atau status.

Fitur Pencarian Multi-Kriteria dapat ditambahkan dengan melakukan traversal seluruh tree $O(n)$. lalu memfilter kategori dan status $O(n)$, dan yang terakhir mengurutkan hasil berdasarkan judul $O(n \log n)$, sehingga kompleksitas untuk Pencarian Multi-Kriteria adalah $O(n \log n)$.

Meskipun begitu, BST ISBN tidak terlalu cocok untuk pencarian multi-kriteria karena tree hanya tersusun berdasarkan ISBN, kategori dan status tidak memiliki index khusus, dan sistem tetap harus memeriksa seluruh node.

Agar query multi-kriteria lebih cepat, sistem dapat menambahkan secondary index menggunakan dictionary/hash map, dengan ini kompleksitas akses kategori + status adalah $O(1)$ karena menggunakan dictionary lookup, pengambilan hasil, lalu jika terdapat K buku yang cocok maka kompleksitasnya $O(K)$, lalu

pengurutan berdasarkan judul memiliki kompleksitas $O(K \log K)$ jika menggunakan algoritma sorting seperti Merge Sort, sehingga total kompleksitas untuk secondary index menggunakan dictionary/hash map yaitu $O(K \log K)$. Kompleksitas $O(K \log K)$ lebih efisien dari $O(n \log n)$, karena K disini hanya sebagai buku dengan kategori dan status yang sesuai.

V. HASIL UJI

1. BANTUAN

```
>>> BANTUAN

DAFTAR PERINTAH:
CARI_BUKU <isbn>
PINJAM <anggota_id> <isbn>
KEMBALIKAN <isbn>
PESAN <anggota_id> <isbn>
BATALKAN_TERAKHIR
REKOMENDASI <isbn>
KATALOG
ANTRIAN <isbn>
BATALKAN_PESANAN <anggota_id> <isbn>
HAPUS_RUSAK <isbn>
LAPORAN_BULAN
KELUAR
```

2. CARI_BUKU

```
>>> CARI_BUKU ISBN-0001

ISBN      : ISBN-0001
Judul     : Digital Vol.1
Pengarang : Penulis-10
Kategori  : Sains
Status    : 1
Peminjam  : 2201
Tgl Pinjam : 17-05-2026 14:45:53
Durasi    : 14 hari
```

```
>>> CARI_BUKU ISBN-0001

ISBN      : ISBN-0001
Judul     : Digital Vol.1
Pengarang : Penulis-10
Kategori  : Sains
Status    : 0
```

3. PINJAM

```
>>> PINJAM 2201 ISBN-0002
Buku ISBN-0002 berhasil dipinjam
[Big-O:  $O(\log n)$ ]
```

```
>>> PINJAM 2205 ISBN-0002
Buku tidak tersedia
```

4. KEMBALIKAN

```
>>> KEMBALIKAN ISBN-0001
Buku ISBN-0001 dikembalikan
[Big-O:  $O(\log n)$ ]
```

5. PESAN

```
>>> PESAN 2202 ISBN-0002
2202 masuk antrian buku ISBN-0002
[Big-O:  $O(\log n)$ ]
```

```
>>> PESAN 2203 ISBN-0003
Buku masih tersedia, gunakan PINJAM
[Big-O:  $O(\log n)$ ]
```

6. ANTRIAN

```
>>> ANTRIAN ISBN-0002
Antrian buku ISBN-0002:
- 2202
- 2206
- 2207
[Big-O:  $O(n)$ ]
```

```
>>> ANTRIAN ISBN-0004
Tidak ada antrian
[Big-O:  $O(n)$ ]
```

7. REKOMENDASI

```
>>> REKOMENDASI ISBN-0001
Rekomendasi untuk ISBN-0001:
ISBN-0002 | freq=2
ISBN-0007 | freq=3
ISBN-0008 | freq=2
[Big-O:  $O(V + E)$ ]
```

```
>>> REKOMENDASI ISBN-0010
Tidak ada rekomendasi
[Big-O:  $O(V + E)$ ]
```

8. KATALOG

```
>>> KATALOG
ISBN-0001 | Digital Vol.1 | Sains
ISBN-0002 | Data Vol.2 | Sains
ISBN-0003 | Python Vol.3 | Fiksi
ISBN-0004 | Elektronika Vol.4 | Teknik
ISBN-0005 | Algoritma Vol.5 | Sains
ISBN-0006 | Fisika Vol.6 | Teknik
ISBN-0007 | Python Vol.7 | Teknik
ISBN-0008 | Sinyal Vol.8 | Sains
ISBN-0009 | Digital Vol.9 | Sains
ISBN-0010 | Sinyal Vol.10 | Seni
ISBN-0011 | Kontrol Vol.11 | Sejarah
ISBN-0012 | Sistem Vol.12 | Teknik
ISBN-0013 | Fisika Vol.13 | Sejarah
ISBN-0014 | Elektronika Vol.14 | Sains
ISBN-0015 | Sinyal Vol.15 | Seni
ISBN-0016 | Python Vol.16 | Sains
ISBN-0017 | Python Vol.17 | Teknik
```

9. BATALKAN TERAKHIR

```
>>> PINJAM 2201 ISBN-0010
Buku ISBN-0010 berhasil dipinjam
[Big-O:  $O(\log n)$ ]

>>> CARI_BUKU ISBN-0010

ISBN      : ISBN-0010
Judul     : Sinyal Vol.10
Pengarang : Penulis-18
Kategori  : Seni
Status    : 1
Peminjam  : 2201
Tgl Pinjam : 17-05-2026 15:18:36
Durasi    : 14 hari

>>> BATALKAN_TERAKHIR
Undo: Transaksi 8 dibatalkan
[Big-O:  $O(\log n)$ ]

>>> CARI_BUKU ISBN-0010

ISBN      : ISBN-0010
Judul     : Sinyal Vol.10
Pengarang : Penulis-18
Kategori  : Seni
Status    : 0
```


10. BATALKAN PESANAN

```
>>> PESAN 2202 ISBN-0010
2202 masuk antrian buku ISBN-0010
[Big-O:  $O(\log n)$ ]

>>> ANTRIAN ISBN-0010
Antrian buku ISBN-0010:
- 2202
[Big-O:  $O(n)$ ]

>>> BATALKAN_PESANAN 2202 ISBN-0010
Pesanan 2202 untuk buku ISBN-0010 berhasil dibatalkan.
[Big-O:  $O(n)$ ]

>>> ANTRIAN ISBN-0010
Tidak ada antrian
[Big-O:  $O(n)$ ]
```

11. HAPUS RUSAK

```
>>> HAPUS_RUSAK ISBN-0011
Buku ISBN-0011 berhasil dihapus dari katalog (Rusak).
[Big-O:  $O(\log n)$ ]

>>> CARI_BUKU ISBN-0011
Buku tidak ditemukan
[Big-O:  $O(\log n)$ ]
```

12. LAPORAN BULANAN

```
>>> LAPORAN_BULAN

=== LAPORAN BULANAN (N Riwayat=4) ===

[Shell Sort - Durasi Descending] Runtime: 0.000026s
ID: 4 | Durasi: 14 hari | Anggota ID: 2205 | ISBN: ISBN-0002
ID: 3 | Durasi: 14 hari | Anggota ID: 2203 | ISBN: ISBN-0003
ID: 2 | Durasi: 14 hari | Anggota ID: 2202 | ISBN: ISBN-0002
ID: 1 | Durasi: 14 hari | Anggota ID: 2201 | ISBN: ISBN-0001

[Merge Sort - Frekuensi Descending] Runtime: 0.000128s
Data Vol.2 | Dipinjam: 2x
Digital Vol.1 | Dipinjam: 1x
Python Vol.3 | Dipinjam: 1x
Elektronika Vol.4 | Dipinjam: 0x
Algoritma Vol.5 | Dipinjam: 0x

Perbandingan Runtime (N=80):
Shell Sort: 0.000026s vs Merge Sort: 0.000128s
[Big-O: Shell  $\sim O(n^{1.5})$ , Merge  $O(n \log n)$ ]
```

13. KELUAR

```
>>> KELUAR
Program selesai
```

VI. KESIMPULAN

Implementasi Smart Library Management & Recommendation System tanpa struktur data bawaan bertujuan untuk mengatasi masalah efisiensi komputasi pada lapisan arsitektur data internal pada perpustakaan modern, selain itu implementasi ini juga sebagai langkah digitalisasi perpustakaan yang masih

melakukan manajemen dan pencatatan transaksi dalam perpustakaan secara konvensional.

Implementasi Smart Library Management & Recommendation System ini menggabungkan beberapa struktur data, diantaranya yaitu Binary Search Tree (BST), Linked List, Stack, Sorting, dan Graph. Kelima struktur data tersebut akan menjadi basis dari CLI yang interaktif untuk membantu manajemen perpustakaan dan pencatatan transaksi perpustakaan.

Sebagian besar fitur-fitur pada CLI masih memiliki kompleksitas worst case $O(n)$, yaitu linear terhadap jumlah data, hal itu dikarenakan sebagian besar fitur berbasis pencarian Binary Search Tree (BST), yang dimana jika tree tersebut skewed maka akan memiliki kompleksitas $O(n)$, sedangkan jika balance maka akan memiliki kompleksitas $O(\log n)$. Meskipun begitu, hanya Shell Sort yang memiliki kompleksitas kuadratik $O(n^2)$ saat melakukan sorting untuk durasi peminjaman buku, sementara algoritma pengurutan lainnya seperti Merge Sort berhasil mengunci efisiensi waktu secara stabil di angka $O(n \log n)$ saat melakukan sorting analisis data buku yang paling banyak dipinjam. Di sisi lain, integrasi struktur data Graph dengan algoritma Breadth First Search (BFS) terbukti memberikan performa yang sangat skalabel sebesar $O(V + E)$ untuk memetakan keterkaitan antar buku sebagai dasar dari fitur rekomendasi.

Secara keseluruhan, pengembangan sistem ini berhasil membuktikan bahwa transisi dari pencatatan konvensional menuju sistem digital berbasis struktur data mandiri (tanpa library bawaan) mampu memberikan kontrol penuh terhadap alokasi memori dan efisiensi komputasi internal perpustakaan. Ketergantungan performa CLI terhadap bentuk pohon BST yang berpotensi menjadi skewed $O(n)$ dan kompleksitas Shell Sort yang kuadratik $O(n^2)$ menjadi catatan evaluasi yang penting.

Dengan demikian, struktur data yang dirancang saat ini sudah jauh lebih adaptif, terorganisir, dan siap untuk ditingkatkan lebih lanjut menggunakan mekanisme auto-balancing tree untuk menjaga kompleksitas dari fitur-fitur di CLI agar tetap $O(\log n)$. Sehingga seluruh operasional utama akan lebih cepat dan efisien di masa depan.

LAMPIRAN

Tabel eksperimen runtime :

Jumlah Koleksi	Waktu Operasi				
	BST Search	Queue	Stack	Shell Sort	Merge Sort
100	$2,78949 \times 10^{(-5)}$	0,00035	0,00025	0,001	$9,53674 \times 10^{(-5)}$
400	$2,81333 \times 10^{(-5)}$	0,00037	0,00026	0,00088	0,00088
800	$2,861022 \times 10^{(-5)}$	0,0004	0,00029	0,00115	0,00093
999	$3,07559 \times 10^{(-5)}$	0,00034	0,00024	0,00094	0,00121

Terminal eksperimen runtime:

1. 100 koleksi

```

=== Eksperimen 1: BST Search ===
Hasil: Jaringan Vol.500
Runtime: 2.7894973754882812e-05

=== Eksperimen 2: Queue ===
Runtime: 0.00035572052001953125

=== Eksperimen 3: Stack ===
Runtime: 0.00025844573974609375

=== Eksperimen 4: Shell Sort ===
Runtime: 0.0010094642639160156

=== Eksperimen 5: Merge Sort ===
Runtime: 9.5367431640625e-05

```

2. 400 koleksi

```

=== Eksperimen 1: BST Search ===
Hasil: Python Vol.500
Runtime: 2.8133392333984375e-05

=== Eksperimen 2: Queue ===
Runtime: 0.00037288665771484375

=== Eksperimen 3: Stack ===
Runtime: 0.0002696514129638672

=== Eksperimen 4: Shell Sort ===
Runtime: 0.0008878707885742188

=== Eksperimen 5: Merge Sort ===
Runtime: 0.0004241466522216797

```

3. 800 koleksi

```
=== Eksperimen 1: BST Search ===  
Hasil: Python Vol.500  
Runtime: 2.86102294921875e-05  
  
=== Eksperimen 2: Queue ===  
Runtime: 0.00040841102600097656  
  
=== Eksperimen 3: Stack ===  
Runtime: 0.0002956390380859375  
  
=== Eksperimen 4: Shell Sort ===  
Runtime: 0.0011510848999023438  
  
=== Eksperimen 5: Merge Sort ===  
Runtime: 0.0009386539459228516
```

4. 999 koleksi

```
=== Eksperimen 1: BST Search ===  
Hasil: Elektronika Vol.500  
Runtime: 3.075599670410156e-05  
  
=== Eksperimen 2: Queue ===  
Runtime: 0.0003452301025390625  
  
=== Eksperimen 3: Stack ===  
Runtime: 0.00024390220642089844  
  
=== Eksperimen 4: Shell Sort ===  
Runtime: 0.0009410381317138672  
  
=== Eksperimen 5: Merge Sort ===  
Runtime: 0.0012195110321044922
```